Chapter 2 — The Math You Need for Machine Learning

Four areas of math appear in every model, library call and interview: **linear algebra** (how data and models are stored and combined), **calculus** (how models learn), **probability** (how models express uncertainty) and **statistics** (how we judge results). A small fifth, **information theory**, explains the most common classification **loss** (the formula scoring how wrong predictions are). Each idea comes with a plain definition, a hand calculation and runnable **NumPy** code (the core Python library for fast calculations on arrays of numbers).

All code is in `code/`. Notation: lowercase (x, w) = vectors, uppercase (X, A) = matrices, ŷ = prediction, Σ = "sum of", · = dot product.

2.1 Vectors

What it is

A **vector** is an ordered list of numbers, such as [2, 5, −3]. In ML **one data point is one vector**: a house with 120 m², 3 bedrooms, 2 bathrooms is x = [120, 3, 2]. Each position is a **feature**; the count of positions is the **dimension**. Geometrically it is an arrow from the origin to a point: a 784-pixel image is a point in 784-D space.

How it works

Operation	Rule	Example, a = [1, 2, 3], b = [4, 5, 6]
Addition	Position by position	a + b = [5, 7, 9]
Scalar (single-number) multiplication	Multiply every entry	2a = [2, 4, 6]
Dot product a · b	Multiply position by position, sum	1·4 + 2·5 + 3·6 = **32**
L2 norm $\|a\|_2$ (length)	√(sum of squares)	√14 ≈ 3.74
L1 norm $\|a\|_1$	Sum of absolute values	6
Max norm $\|a\|_\infty$	Largest absolute value	3
Unit vector	a / $\|a\|_2$	[0.27, 0.53, 0.80]
Cosine similarity	(a · b) / (‖a‖ ‖b‖)	32 / (3.74 · 8.77) ≈ 0.975

The dot product is the most important operation in ML. A *linear model* (prediction = weighted sum of features) predicts $\hat{y} = w \cdot x + b = w_1x_1 + w_2x_2 + w_3x_3 + b$. With w = [2, 10, 5] and x = [120, 3, 2]: w · x = 240 + 30 + 10 = **280**.

Geometrically a · b = ‖a‖ ‖b‖ cos θ (θ = angle between the arrows): large when vectors point the same way, zero when perpendicular, negative when opposite. **Cosine similarity** keeps only the angle; search and recommender systems use it to compare **embeddings** (learned vectors representing meaning).

Norms measure size; the distance between points is $\|a - b\|_2$. L1 and L2 norms of the *weights* are **regularisation** penalties (extra loss terms discouraging large weights, to reduce overfitting): Lasso regression uses L1, Ridge uses L2.

Code

`code/linear_algebra.py` :

```
house = np.array([120.0, 3.0, 2.0])
w = np.array([2.0, 10.0, 5.0])
print("dot w.x =", w @ house)                    # @ is the dot / matrix product
a, b = np.array([1.0, 0.0]), np.array([1.0, 1.0])
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
v = np.array([3.0, -4.0])
print(np.linalg.norm(v), np.linalg.norm(v, 1), np.linalg.norm(v, np.inf))
```

Output:

```
x = [120.   3.   2.] | w = [ 2. 10.  5.]
x + w = [122.  13.   7.] | 2 * x = [240.   6.   4.]
dot w.x = 280.0 (= 2*120 + 10*3 + 5*2)
cosine similarity of [1,0] and [1,1] = 0.707 (angle 45 deg)
L2 norm of [3,-4] = 5.0 | L1 norm = 7.0 | max norm = 4.0
unit vector = [ 0.6 -0.8]
```

Common mistakes

- `a * b` in NumPy is element-wise (returns a vector); `a @ b` / `np.dot` is the dot product.
- Comparing distances when features have wildly different scales (m² in hundreds, bedrooms in units): the large one dominates. Standardise first (§2.8).

Self-check

1. Why cosine over dot product for documents? → It ignores length (document size) and compares direction (content).

TL;DR

A data point is a vector. The dot product is how a linear model combines features; norms measure size and distance; cosine similarity measures direction.

2.2 Matrices and matrix multiplication

What it is

A **matrix** is an m × n grid of numbers. **A dataset is a matrix** (rows = examples, columns = features); so are weights (a layer mapping 3 inputs to 2 outputs has a 2 × 3 matrix).

How matrix multiplication works

A (m × n) times B (n × p): inner sizes must match; the result is m × p; entry (i, j) = **row i of A · column j of B**.

```
A = | 1 2 |     B = | 5 6 |
    | 3 4 |         | 7 8 |

(A B)[0,0] = 1·5 + 2·7 = 19      (A B)[0,1] = 1·6 + 2·8 = 22
(A B)[1,0] = 3·5 + 4·7 = 43      (A B)[1,1] = 3·6 + 4·8 = 50
```

Order matters: **AB ≠ BA** in general (here BA = [[23, 34], [31, 46]]).

Batch prediction is one matrix product: ŷ = Xw + b, each row of X dotted with w. Libraries run this as optimised CPU/GPU code (a GPU is a graphics chip doing thousands of simple calculations in parallel), so ML code avoids Python loops. A neural-network layer computes h = activation(Wx + b) (an *activation function* is a simple non-linear function, e.g. "replace negatives with 0"; without it stacked layers collapse into one linear map). Deep networks are long chains of matrix multiplications, which is why GPUs power deep learning.

Transpose, identity, inverse, determinant

Concept	Definition	Example / use
Transpose A^T	Swap rows and columns (n × m)	[[1, 2], [3, 4]]^T = [[1, 3], [2, 4]]; in gradients and covariance (how features vary together)
Identity I	1s on diagonal, 0s elsewhere; AI = A	The "1" of matrices
Inverse A^{-1}	A A^{-1} = I (square matrices with det ≠ 0 only)	"Undoes" A; solves linear equations
Determinant det(A)	How A scales area/volume; 0 = squashes space flat, no inverse	2 × 2: ad − bc = 1·4 − 2·3 = **−2**

2 × 2 inverse: A^{-1} = (1/det) · [[d, −b], [−c, a]] = (1/−2) · [[4, −2], [−3, 1]] = [[−2, 1], [1.5, −0.5]].

Solving equations. $x_1 + 2x_2 = 5$, $3x_1 + 4x_2 = 11$ is Ax = c, so x = A^{-1}c = [1, 2]. In code use `np.linalg.solve(A, c)`, not the inverse: faster and numerically more accurate. The **closed-form** (direct formula, no iterative search) linear-regression solution, the **normal equation** w = $(X^TX)^{-1}X^Ty$, is solved the same way; scikit-learn's `LinearRegression` uses a **least-squares** solver (finds weights minimising the sum of squared errors) based on matrix decompositions (rewriting a matrix as a product of simpler ones), not an explicit inverse.

Code

```
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
A @ B            # matrix product
A * B            # element-wise product (different!)
A.T              # transpose
np.linalg.det(A); np.linalg.inv(A)
np.linalg.solve(A, np.array([5.0, 11.0]))
X @ w + b        # predictions for all houses at once
```

Output:

```
X shape: (4, 3) | predictions X @ w + b = [330. 235. 505. 270.]
A @ B =
 [[19 22]
 [43 50]]
B @ A =
 [[23 34]
 [31 46]]
(A @ B != B @ A: order matters)
A * B (element-wise) =
 [[ 5 12]
 [21 32]]
A.T =
 [[1 3]
 [2 4]]
det(A) = -2.0
inv(A) =
 [[-2.   1. ]
 [ 1.5 -0.5]]
A @ inv(A) =
 [[1. 0.]
 [0. 1.]]
solve A x = [5, 11] -> [1. 2.]
layer output W @ x = [2.2 0.3]
```

Common mistakes

- **Shape errors.** Write shapes first: (4 × 3)(3,) → (4,). Mismatched inner sizes mean a missing transpose or the wrong matrix.

- Inverting nearly singular matrices (det ≈ 0), e.g. two features almost copies of each other (*multicollinearity*): results blow up. Regularisation or `solve` / `lstsq` helps.

In the interview

> **30-second answer:** "A dataset is a matrix with examples as rows and features as columns, and a linear model's predictions for the whole dataset are one matrix-vector product, Xw + b. Neural network layers are matrix multiplications followed by nonlinearities (activation functions), which is why GPUs, which are very fast at matrix math, dominate deep learning."

- *Is matrix multiplication commutative?* — No; AB ≠ BA in general, and shapes may not even allow BA.
- *When does a matrix have no inverse?* — Not square, or det = 0 (columns linearly dependent: one is a weighted sum of others).

Self-check

1. det([[2, 1], [4, 2]])? → 0: no inverse (rows are multiples).

TL;DR

A dataset is a matrix; predictions are Xw + b. Multiplication = rows dotted with columns, order matters. Transpose swaps axes; the inverse exists only when det ≠ 0.

2.3 Eigenvalues, eigenvectors and SVD (intuition)

What it is

A matrix usually rotates and stretches a vector. An **eigenvector** of square A is a direction A only stretches (or flips): A v = λ v; λ is the **eigenvalue**.

Example: A = [[2, 1], [1, 2]], v = [1, 1]: A v = [3, 3] = 3v → eigenvalue 3. The other eigenvector is [1, −1] with eigenvalue 1.

Singular value decomposition (SVD) works for any matrix: $M = U \Sigma V^T$, "rotate (V^T), stretch along axes (Σ), rotate (U)". The **singular values** on Σ's diagonal, largest first, say how much each direction matters.

Why it matters

- **PCA (principal component analysis)**, the standard **dimensionality reduction** method (many features → a few that keep most information), takes eigenvectors of the data's **covariance matrix** (how every pair of features varies together) — equivalently the SVD of the centred data (column means subtracted). Top eigenvectors are the directions of most variation; eigenvalues say how much variance (§2.6) each captures.
- **Compression/denoising:** keeping the top k singular values gives the best **rank-k approximation** (closest matrix built from only k independent directions) — Eckart–Young theorem.
- **Recommenders:** matrix factorisation, made famous by the Netflix Prize (2006–2009), approximates the user × movie ratings matrix as a product of two thin matrices, an idea closely related to SVD.
- **PageRank**, Google's original ranking, is the leading eigenvector of a matrix built from the web's link graph.

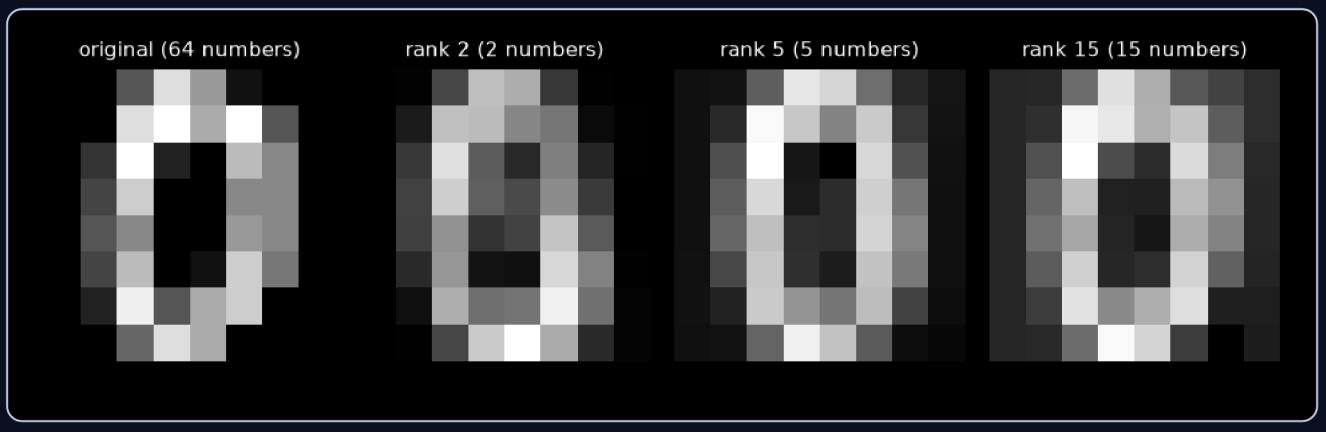
Code

```
vals, vecs = np.linalg.eigh(np.array([[2.0, 1.0], [1.0, 2.0]]))  # eigh: symmetric matrices
data = rng.multivariate_normal([0, 0], [[3, 2], [2, 2]], size=2000)
vals, vecs = np.linalg.eigh(np.cov(data, rowvar=False))           # PCA in two lines
U, s, Vt = np.linalg.svd(pixels - pixels.mean(0), full_matrices=False)
approx = (x @ Vt[:k].T) @ Vt[:k] + pixels.mean(0)                  # keep top-k directions
```

Output:

```
eigenvalues: [1. 3.]
eigenvectors (columns):
 [[-0.707  0.707]
 [ 0.707  0.707]]
A @ v = [2.121 2.121] = 3 * v = [2.121 2.121]
covariance eigenvalues: [0.433 4.584] -> share of variance on main axis: 0.914
singular values: [3.464 3.162]
rebuild U @ diag(s) @ Vt =
 [[ 3.  1.  1.]
 [-1.  3.  1.]]
top  5 of 64 singular values keep 54.5% of the variance
top 10 of 64 singular values keep 73.8% of the variance
top 20 of 64 singular values keep 89.4% of the variance
```

Eigenvectors come normalised ([1, 1] → [0.707, 0.707]). In the 2-D cloud one direction holds 91% of the variance; on 8 × 8 digit images (64 pixels), 20 directions keep 89%:



Common mistakes

- PCA without centring (and usually standardising).
- `np.linalg.eig` on symmetric matrices like covariance; `eigh` is faster, more stable and returns real, sorted eigenvalues.

TL;DR

Eigenvectors are directions a matrix only stretches; eigenvalues are the amounts. SVD factors any matrix into rotate–stretch–rotate. They power PCA, compression and recommenders.

2.4 Derivatives, partial derivatives and gradients

What it is

- A **derivative** f'(x) is the slope at a point. f(x) = x² → f'(x) = 2x; at x = 3, nudging x by 0.001 raises f by ≈ 0.006.
- A **partial derivative** $\partial f/\partial x$ is the slope along one input, others held fixed.
- The **gradient** ∇f is the vector of all partials; it points to steepest *increase*, its negative downhill.

Why it matters

The **loss** is a function of the parameters; its gradient tells each parameter which way and how much to move to reduce error. That is "learning" for most models, from linear regression to GPT-scale networks.

How it works: rules you actually need

Rule	Formula	Example
Constant	$(c)' = 0$	$(7)' = 0$
Power	$(x^n)' = n\,x^{n-1}$	$(x^3)' = 3x^2$
Sum	$(f + g)' = f' + g'$	$(x^2 + 3x)' = 2x + 3$
Constant multiple	$(c\,f)' = c\,f'$	$(5x^2)' = 10x$
Exponential / log	$(e^x)' = e^x$, $(\ln x)' = 1/x$	
Chain rule	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	$((3x + 1)^2)' = 2(3x + 1) \cdot 3$
Sigmoid (S-curve squashing any number into 0–1)	$\sigma(z) = 1/(1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$	$\sigma(0) = 0.5$, $\sigma'(0) = 0.25$

Partials, worked. $f(x, y) = x^2y + 3y$ at $(2, 5)$: $\partial f/\partial x = 2xy =$ **20**; $\partial f/\partial y = x^2 + 3 =$ **7**; $\nabla f = [20, 7]$.

The chain rule (the engine of backpropagation)

For nested functions, multiply local slopes along the chain. A one-weight **neuron** (weighted input, then activation) with squared error: $z = w \cdot x \to \hat{y} = \sigma(z) \to L = (\hat{y} - y)^2$, with $x = 2$, $y = 1$, $w = 0.5$:

1. Forward: $z = 1.0$, $\hat{y} = \sigma(1) = 0.7311$, $L = 0.0723$
2. Local slopes: $dL/d\hat{y} = 2(\hat{y} - y) = -0.5379$; $d\hat{y}/dz = \hat{y}(1 - \hat{y}) = 0.1966$; $dz/dw = x = 2$
3. Chain: $dL/dw = -0.5379 \times 0.1966 \times 2 =$ **-0.2115**

Negative → increase w to reduce the loss ($\hat{y}$ is below the target). **Backpropagation** is this, applied efficiently to millions of weights from the output layer backwards, reusing shared intermediate products. PyTorch and similar libraries do it automatically (*autograd*, automatic differentiation: record each operation, derive gradients for you).

Code: check every derivative numerically

`code/calculus.py` compares hand formulas with the **central difference** $f'(x) \approx (f(x + h) - f(x - h)) / 2h$, a standard "gradient check":

```
def numerical_derivative(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

# gradient of MSE for linear regression, vectorised: (2/n) X^T (Xw - y)
grad = 2 / len(yv) * X.T @ (X @ wv - yv)
```

Output:

```
d/dx x^2 at x=3: numeric 6.0000, exact 6
gradient of x^2*y + 3y at (2,5): numeric [20.  7.] exact [20. 7.]
d/dx (3x+1)^2 at x=2: numeric 42.0000, exact 42
z=1.0, y_hat=0.7311, dL/dyhat=-0.5379, dyhat/dz=0.1966, dz/dw=2.0
dL/dw via chain rule -0.21151 | numeric -0.21151
MSE gradient: formula [-4.         -9.33333333] | numeric [-4.     -9.3333]
```

The MSE gradient (MSE = mean squared error), seen everywhere: for $L(w) = (1/n)\,\Sigma\,(x_i \cdot w - y_i)^2$, $\nabla L = (2/n)\,X^T(Xw - y)$ — each example's error times its features, averaged. Here X has a column of 1s (so $w[0]$ is the **bias**, the constant b shifting every prediction), $y = [2, 4, 6]$, $w = [0, 1]$: errors $[-1, -2, -3]$ give $[2/3 \cdot (-6), 2/3 \cdot (-14)] = [-4, -9.33]$.

Common mistakes

- Dropping the inner function's factor in the chain rule (the "· 3" in $((3x + 1)^2)'$).
- The gradient points **uphill**; we step along **minus** the gradient.

Self-check

1. Why does σ'(z) get tiny for large |z|? → σ flattens near 0 and 1, so σ(1 − σ) → 0: the "vanishing gradient" problem in deep nets.

TL;DR

Derivative = slope; partial = slope along one input; gradient = vector of partials pointing uphill. The chain rule multiplies slopes through nested functions — that is backpropagation.

2.5 Gradient descent

What it is

Gradient descent minimises a function: start somewhere, compute the gradient, step the opposite way, repeat (walking down a foggy hill by the steepest slope underfoot). Update: **$w \leftarrow w - \eta \cdot \nabla L(w)$**, where η (eta) is the **learning rate**, a **hyperparameter** (set before training, not learned) controlling step size.

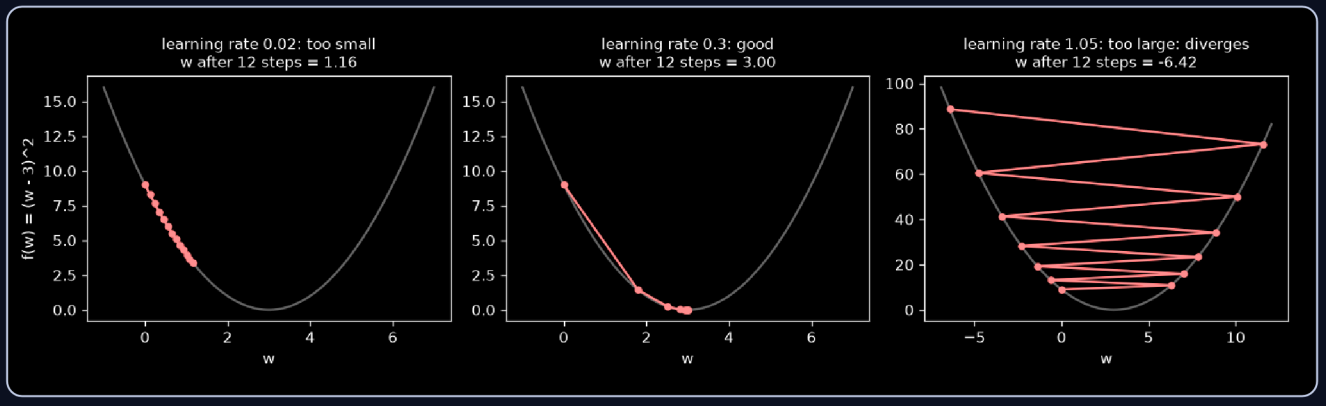
Worked by hand

Minimise $f(w) = (w - 3)^2$ (minimum at 3); $f'(w) = 2(w - 3)$; start $w = 0$, $\eta = 0.1$.

Step	w	f(w)	gradient 2(w − 3)	new w = w − 0.1 · gradient
0	0.0000	9.0000	−6.0000	0 + 0.6 = 0.6000
1	0.6000	5.7600	−4.8000	0.6 + 0.48 = 1.0800
2	1.0800	3.6864	−3.8400	1.4640
3	1.4640	2.3593	−3.0720	1.7712
4	1.7712	1.5099	−2.4576	2.0170

Each step closes 20% of the remaining distance ($w - 3$ is multiplied by $1 - 2\eta = 0.8$); steps shrink as the slope flattens.

The learning rate



- **Too small (0.02):** after 12 steps w is only 1.16.
- **Good (0.3):** reaches 3.00 in 12 steps.
- **Too large (1.05):** each step overshoots further; w diverges (−6.42 and growing). Here any $\eta > 1$ diverges because $|1 - 2\eta| > 1$.

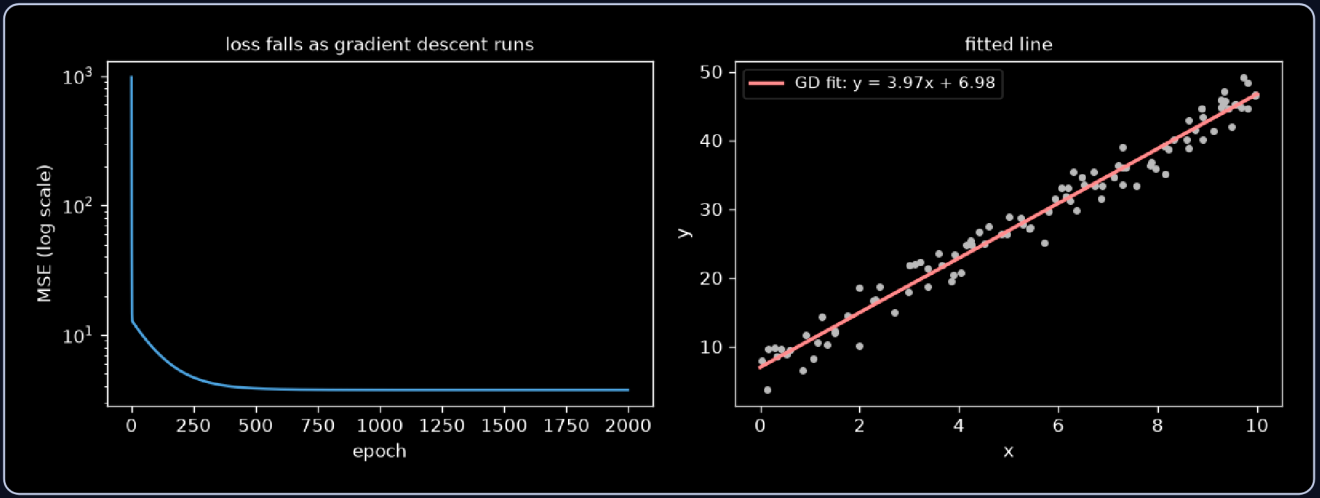
Gradient descent for linear regression

`code/gradient_descent.py` fits $y = w \cdot x + b$ to 100 noisy points from $y = 4x + 7$:

```python
w, b, lr = 0.0, 0.0, 0.01
for epoch in range(2000):
    err = (w * x + b) - y
    dw = 2 * np.mean(err * x)      # dMSE/dw
    db = 2 * np.mean(err)          # dMSE/db
    w -= lr * dw
    b -= lr * db
```

Output:

```
epoch    0: w=0.000 b=0.000 MSE=976.742
epoch    1: w=3.885 b=0.575 MSE=51.369
epoch   10: w=4.820 b=0.966 MSE=12.240
epoch  100: w=4.533 b=3.007 MSE=7.464
epoch 1999: w=3.974 b=6.984 MSE=3.760
scikit-learn closed form: w=3.974 b=6.985
```



It matches scikit-learn's exact solver. w overshoots early while b creeps up: the **loss surface** (loss as a landscape over parameter values) is a long narrow valley because x (0–10) and the constant 1 have different scales. **Standardising features** rounds the valley and speeds convergence.

Variants

Variant	Gradient on	Pros	Cons
Batch GD	All n examples per step	Smooth, exact gradient	Slow per step on big data
Stochastic GD (SGD)	One random example	Very cheap steps; noise can escape shallow **local minima** (dips lowest only nearby)	Noisy; needs a decaying learning rate
Mini-batch GD	Small batch (e.g. 32–512)	Best of both; uses GPU parallelism	Extra hyperparameter (batch size)

One pass over the training set is an **epoch**. Deep-learning **optimisers** (update rules) such as Momentum, RMSProp and Adam are mini-batch GD with adaptive step sizes.

Convex vs non-convex. A convex (bowl-shaped) loss — MSE for linear regression, log loss for logistic regression — has no separate local minima, so GD with a suitable η reaches the global minimum. Neural-network losses are non-convex with many local minima and **saddle points** (flat spots curving up in some directions, down in others); mini-batch methods still find good solutions in practice.

When not to use it

A closed-form solver exists and data fits in memory (small linear regression). GD shines with large data, complex models or no closed form.

Common mistakes

- Unscaled features → slow, zig-zagging convergence.
- η too large → loss grows or becomes NaN ("Not a Number", from invalid maths like ∞ − ∞). First fix: divide η by 10.
- Not shuffling between SGD epochs; stopping too early (plot the loss curve).

In the interview

> **30-second answer:** "Gradient descent minimises a loss by repeatedly moving parameters a small step opposite the gradient: $w \leftarrow w - \eta \nabla L$. The learning rate η controls step size; too small is slow, too large diverges. In practice we use mini-batch stochastic gradient descent, often with an adaptive optimiser like Adam, and we scale features so it converges quickly."

- *Why mini-batches?* — Cheaper than full-batch, less noisy than single examples, maps well to GPUs.
- *Always the global minimum?* — Guaranteed only for convex problems with a suitable η.

Self-check

1. One step on $f(w) = w^2$ from $w = 4$, $\eta = 0.25$? → gradient 8, $w = 2$.

TL;DR

$w \leftarrow w - \eta \nabla L$. Pick η carefully, scale features, use mini-batches, watch the loss curve.

2.6 Random variables and distributions

What it is

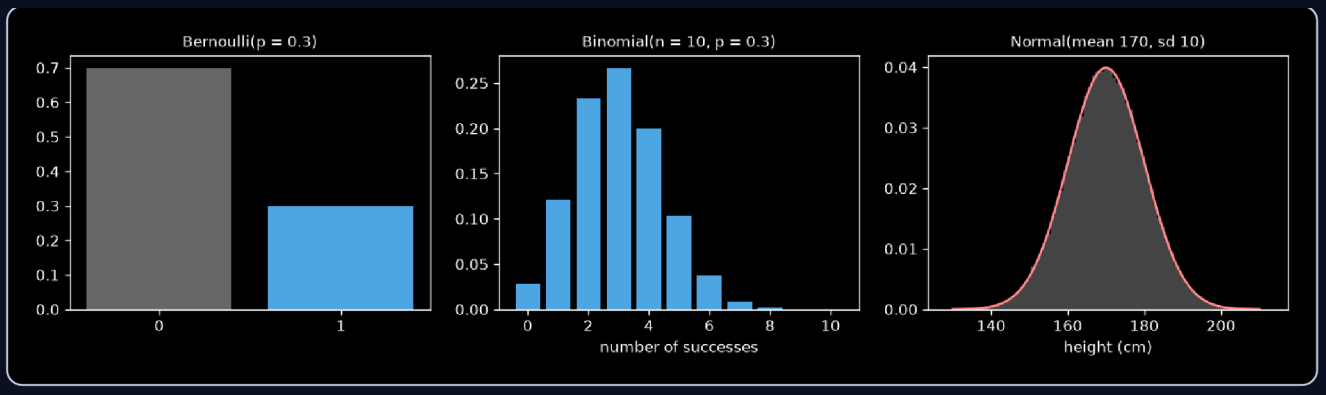
Probability is a number from 0 (impossible) to 1 (certain), e.g. P(spam) = 0.2. A **random variable** takes its value from a random process (die roll, spam or not, tomorrow's temperature): **discrete** (countable values) or **continuous** (any value in a range). A **probability distribution** says how likely each value is: a *probability mass function (PMF)* for discrete, a *probability density function (PDF)* for continuous (probabilities are areas under it).

The three distributions you'll meet most

Distribution	Models	Parameters	Mean	Variance	ML use
Bernoulli	One yes/no trial	p = P(1)	p	p(1 − p)	Binary label; logistic regression outputs p
Binomial	Successes in n independent trials	n, p	np	np(1 − p)	Clicks out of n ad impressions; A/B tests (comparing two versions on random user groups)
Normal (Gaussian)	Values clustered symmetrically around a mean	μ (mean), σ (std)	μ	σ^2	Measurement noise, heights, linear-regression errors; weight initialisation

Binomial: P(exactly 3 of 10, p = 0.3) = C(10, 3) · 0.3^3 · 0.7^7 = 120 · 0.027 · 0.0824 ≈ **0.2668** (C(10, 3) = 120 ways to pick the 3 successes).

68–95–99.7 rule: ~68% of normal values lie within 1σ, 95% within 2σ, 99.7% within 3σ. Heights ~ Normal(170, 10): P(height > 190) = P(z > 2) ≈ **2.3%**.



Expectation and variance

- **Expectation (mean)** $E[X] = \Sigma\ x \cdot P(x)$, the long-run average. Fair die: **3.5**.
- **Variance** $Var[X] = E[(X - E[X])^2]$, the average squared distance from the mean. Die: 17.5/6 ≈ **2.917**.
- **Standard deviation** $\sigma = \sqrt{Var}$, in X's units (die ≈ 1.71).

Rules: $E[aX + b] = aE[X] + b$; $Var[aX + b] = a^2Var[X]$; independent X, Y: $Var[X + Y] = Var[X] + Var[Y]$.

Code

code/probability.py simulates each distribution against theory:

```
flips = rng.random(100_000) < 0.3                # Bernoulli(0.3)
draws = rng.binomial(10, 0.3, 100_000)           # Binomial(10, 0.3)
heights = rng.normal(170, 10, 100_000)           # Normal(170, 10)
stats.binom.pmf(3, 10, 0.3); 1 - stats.norm.cdf(190, 170, 10)
```

Output:

```
Bernoulli(0.3): sample mean 0.299 (theory 0.3), variance 0.210 (theory p(1-p) = 0.21)
Binomial(10, 0.3): P(X=3) = C(10,3) 0.3^3 0.7^7 = 0.2668 | scipy 0.2668
  mean 2.994 (theory np = 3.0), variance 2.093 (theory np(1-p) = 2.1)
Normal(170, 10): within 1 sd 0.683, within 2 sd 0.954 (theory 0.683, 0.954)
  P(height > 190) = 0.0228  (z = (190-170)/10 = 2)
die: E[X] = 3.500, Var[X] = 2.917 | simulated mean 3.510, var 2.925
```

Common mistakes

- Reading a PDF value as a probability: for continuous variables P(X = exactly 170) = 0; only ranges have probability.
- Assuming normality. Incomes, session lengths and prices are usually **right-skewed** (mostly small, long tail of large values); a log transform often helps.

Self-check

1. Mean and variance of Binomial(100, 0.1)? → 10 and 9.
2. Var[3X] if Var[X] = 2? → 18.

TL;DR

Bernoulli = one yes/no, binomial = count of yeses, normal = symmetric bell. Expectation = long-run average; variance = average squared spread.

2.7 Conditional probability and Bayes' theorem

What it is

Conditional probability P(A | B) = P(A and B) / P(B): the probability of A given B happened. Of 1,000 emails, 200 are spam; 120 spam and 40 normal emails contain "free": P(spam | free) = 120 / 160 = **0.75**,

versus the base rate 0.2. A and B are **independent** if P(A | B) = P(A).

Bayes' theorem: P(A | B) = P(B | A) · P(A) / P(B), i.e. **posterior = likelihood × prior / evidence**:

- **Prior** P(A): belief before the data.
- **Likelihood** P(B | A): probability of the data if A is true.
- **Evidence** P(B) = P(B | A)P(A) + P(B | not A)P(not A).
- **Posterior** P(A | B): updated belief.

Worked example: a medical test

Disease prevalence 1%; the test catches 90% of sick people (**sensitivity** P(+ | sick) = 0.90) and flags 5% of healthy ones (false-positive rate 0.05, **specificity** 0.95). You test positive:

1. Prior P(sick) = 0.01
2. Evidence P(+) = 0.90 · 0.01 + 0.05 · 0.99 = **0.0585**
3. Posterior P(sick | +) = 0.009 / 0.0585 = **0.154**

Only ~15%. Of 10,000 people, 100 are sick (90 test positive) and 9,900 healthy (495 test positive): only 90 of 585 positives are sick. This **base-rate fallacy** matters for every rare-event classifier (fraud, rare disease, intrusion detection). Bayes chains: after a second independent positive, the old posterior is the new prior: 0.9 · 0.154 / (0.9 · 0.154 + 0.05 · 0.846) ≈ **0.766**.

```
evidence = sens * prior + fpr * (1 - prior)
posterior = sens * prior / evidence
```

Output (including a 1,000,000-person simulation):

```
P(free) = 0.16, P(spam | free) = 120/160 = 0.75
P(positive) = 0.9*0.01 + 0.05*0.99 = 0.0585
P(disease | positive) = 0.009 / 0.0585 = 0.154
simulation: P(disease | positive) = 0.153
after a second independent positive test: 0.766
```

Real-world examples

- **Naive Bayes spam filters** apply Bayes assuming words are independent given the class; Paul Graham's 2002 essay "A Plan for Spam" popularised them.
- **Calibration:** a fraud score of 0.9 should mean 90% of such cases are fraud; Bayes explains why scores trained on balanced samples must be adjusted when real fraud is rare.

Common mistakes

- Confusing P(A | B) with P(B | A): P(+ | sick) = 0.9 is not P(sick | +) = 0.15.
- Ignoring the prior (base rate).

In the interview

30-second answer: "Bayes' theorem updates a prior belief with evidence: P(A|B) = P(B|A)P(A)/P(B). A classic example is a test with 90% sensitivity and 5% false positives for a disease with 1% prevalence: a positive result gives only about a 15% chance of disease, because the healthy population is so much larger. It underlies Naive Bayes and explains why precision (the share of flagged cases that really are positive) is low for rare-event classifiers."

- *What's "naive" in Naive Bayes?* — Features assumed conditionally independent given the class.
- *Relation to precision?* — P(sick | positive) is exactly the test's precision.

TL;DR

P(A | B) = P(A and B) / P(B). Posterior ∝ likelihood × prior. Rare events + imperfect tests = many false alarms.

2.8 Descriptive statistics and standardisation

What it is

Statistics summarises data and quantifies how far conclusions can be trusted.

Measure	Definition	Robust to outliers?
Mean	Sum ÷ count	No
Median	Middle sorted value	Yes
Mode	Most frequent value	Yes
Variance	Average squared distance from the mean	No
Standard deviation	√variance, original units	No
Percentiles / IQR	Value below which a % of data falls; IQR = 75th − 25th percentile	Yes

Worked example. Salaries ($1000s): 40, 42, 45, 47, 50, 52, 55, 300 (one CEO). Mean = **78.9**, median = **48.5**: one outlier drags the mean far above a typical salary. **Population vs sample std:** dividing by n (NumPy default `ddof=0`) describes this exact group; n − 1 (`ddof=1` , the pandas default) gives an unbiased variance estimate for a sample of a larger population; negligible difference for large n.

Standardisation (z-scores): z = (x − μ)/σ gives mean 0, std 1; z = "standard deviations from the mean". For x = [2, 4, 4, 4, 5, 5, 7, 9]: μ = 5, σ = 2, so 9 → **2.0**, 2 → **−1.5**. It stops features dominating through units, speeds up gradient descent, and is essential for distance-based methods (k-NN, k-means, SVMs) and regularised models. scikit-learn's `StandardScaler` learns μ and σ from training data only.

Output of `code/statistics_basics.py`:

```
mean 78.9, median 48.5
population std (ddof=0) 83.7, sample std (ddof=1) 89.5
without the outlier: mean 47.3, median 47.0
x mean 5.0, std 2.0 -> standardised z = [-1.5 -0.5 -0.5 -0.5  0.   0.   1.   2. ]
z mean 0.0, z std 1.0
```

Self-check

1. Why fit the scaler on training data only? → Test data in μ and σ leaks test information.

TL;DR

Mean and std are pulled by outliers; median and IQR are robust. Standardise to mean 0, std 1 before distance- or gradient-based models.

2.9 Correlation vs causation

What it is

Correlation (Pearson's r, −1 to +1) measures how strongly two variables move together *in a straight line*: +1 perfect increasing line, −1 perfect decreasing, 0 no *linear* relationship. **Causation** means changing one actually changes the other. Correlation without causation comes from:

1. **Confounder:** a third variable drives both (hot weather raises ice-cream sales and swimming, hence drownings).
2. **Reverse causation:** the arrow runs the other way.
3. **Coincidence:** with enough variables some correlate by chance.
4. **Selection bias:** the data collection creates the pattern.

```
temp = rng.uniform(10, 35, 365)
ice_cream = 20 * temp + noise;  drownings = 0.3 * temp + noise
np.corrcoef(ice_cream, drownings)          # strong correlation
# remove temperature's effect from both, then correlate what is left
```

Output:

```
corr(ice cream, drownings) = 0.77
corr after controlling for temperature = -0.06
corr(x, x^2) = -0.00  (perfect dependence, zero LINEAR correlation)
```

Controlling for temperature removes the relationship. Conversely $y = x^2$ is fully determined by x yet $r = 0$ because the relationship is not a line.

Why it matters for ML

Models find correlations, not causes: fine for *prediction* (umbrella sales predict rain), dangerous for *decisions* ("sell umbrellas to cause rain"). Causal effects need **randomised experiments (A/B tests)** or causal-inference methods.

Self-check

1. More hospitals, more deaths. Causal? → Likely confounded by population size.
2. $r = 0$ means independent? → Not necessarily; there may be a non-linear relationship.

TL;DR

Correlation measures linear co-movement and can come from confounders, reverse causation or chance. Use experiments to establish cause.

2.10 Sampling and the central limit theorem

What it is

A **population** is everything you care about; a **sample** is what you measure. A sample **statistic** (e.g. sample mean) estimates a population **parameter**. Samples must be **random** and **representative**; a biased sample gives biased estimates however large (Chapter 1's *Literary Digest* poll).

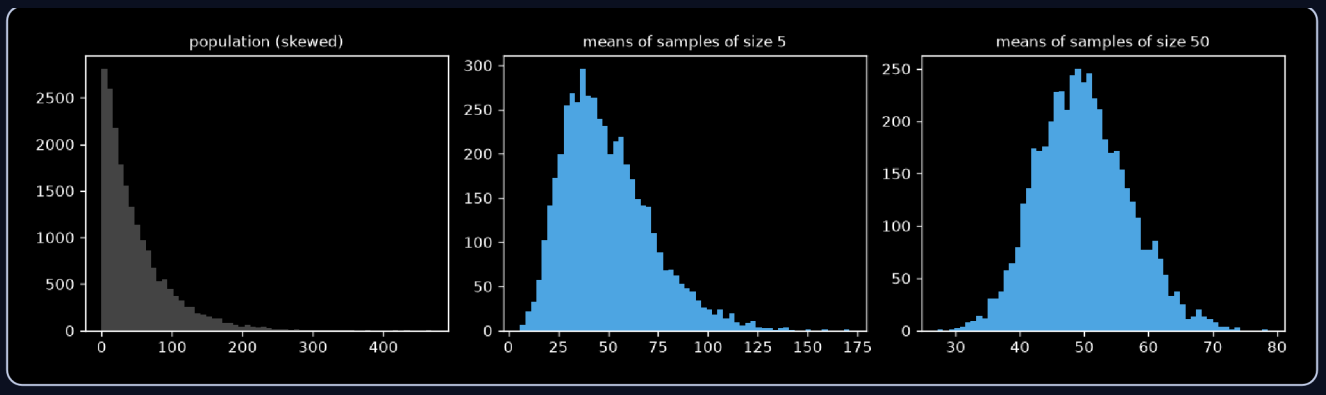
Central limit theorem (CLT): means of many random samples of size n are approximately normal around the true mean, even for a skewed population, if n is reasonably large. Their spread, the **standard error**, is **$\sigma/\sqrt{n}$**: quadrupling n halves the uncertainty.

Code

```
population = rng.exponential(scale=50, size=1_000_000)       # skewed
means = rng.choice(population, size=(5000, n)).mean(axis=1)  # 5000 samples of size n
```

Output:

```
population mean 49.99, std 50.07
n= 5: mean of sample means 49.81, std of sample means 22.48 (theory sigma/sqrt(n) = 22.39)
n=50: mean of sample means 49.89, std of sample means 7.11 (theory sigma/sqrt(n) = 7.08)
```



At n = 50 the means form a bell despite the skew — why normal-based CIs and t-tests (§2.11) work for averages of real data, and why larger test sets give more reliable accuracy.

TL;DR

Sample means cluster normally around the truth with spread σ/√n. Random, representative samples matter more than big ones.

2.11 Hypothesis tests, p-values and confidence intervals

What it is

A **hypothesis test** asks whether an effect could plausibly be noise:

1. State the **null hypothesis H_0** (no effect) and **alternative H_1**.
2. Choose **significance level α** beforehand, usually 0.05.
3. Compute a **test statistic** (e.g. t = difference in means / its standard error).
4. **p-value**: *if H_0 were true*, the probability of a result at least this extreme.
5. p < α → **reject H_0** ("statistically significant"); otherwise **fail to reject** (not proof of H_0).

Error	Meaning	Probability
Type I (false positive)	Reject a true H_0	α
Type II (false negative)	Miss a real effect	β; **power** = 1 − β

A **confidence interval (CI)** = estimate ± critical value × standard error (the *critical value* comes from a reference distribution, ≈ 1.96 for 95% with large samples). A 95% CI comes from a method that captures the true value in 95% of repeated experiments.

Worked example: an A/B test

Page load times, 60 users each: old 202.7 ms, new 187.3 ms (−15.4 ms). Welch's t-test (no equal-variance assumption):

```
t, p = stats.ttest_ind(new, old, equal_var=False)
m, se = new.mean(), new.std(ddof=1) / np.sqrt(len(new))
tcrit = stats.t.ppf(0.975, df=len(new) - 1)
```

Output:

```
old mean 202.7, new mean 187.3, diff -15.4 ms
Welch t = -2.72, p-value = 0.0075 -> reject H0 at 0.05
95% CI for new mean: 187.3 +/- 2.001*4.02 = [179.2, 195.3]
accuracy 0.850, bootstrap 95% CI [0.800, 0.900]
coverage of 2000 simulated 95% CIs: 0.949
```

- p = 0.0075: with no real difference, a gap this large would appear < 1% of the time → reject H_0.

- 2.001 is the t critical value for 59 **degrees of freedom** (roughly sample size minus quantities estimated); it approaches 1.96 for large samples.
- **Bootstrap CI** for test accuracy: resample the 200 test predictions with replacement 5,000 times, take the 2.5th and 97.5th percentiles. 85% on 200 examples really means "about 80–90%" — remember this before celebrating a 1-point gain on a small test set.
- 94.9% of 2,000 simulated 95% intervals contained the true mean.

Common mistakes

- **"p = 0.03 means a 3% chance H_0 is true."** No: p assumes H_0 and measures how surprising the data is.
- **Significant ≠ important.** Huge samples make trivial differences significant; report the **effect size** ("15 ms faster") with its CI.
- **Multiple comparisons / p-hacking:** testing 20 useless features at α = 0.05 yields ~one "significant" by chance. Correct (Bonferroni: α/20) or pre-register (write down the test before seeing data).
- "Peeking": stopping an A/B test as soon as p < 0.05 inflates false positives.

In the interview

> **30-second answer:** "A p-value is the probability of seeing data at least as extreme as ours if the null hypothesis were true. If it's below a pre-chosen α, like 0.05, we reject the null. It's not the probability the null is true, and significance isn't practical importance, so I report effect sizes with confidence intervals and correct for multiple comparisons."

- *Statistical power?* — Probability of detecting a real effect; rises with sample size and effect size.
- *Comparing two models' accuracy?* — Same test set, paired test (e.g. McNemar's, comparing examples one model gets right and the other wrong) or bootstrap the difference.

Self-check

1. p = 0.2, α = 0.05? → Fail to reject H_0; consistent with no effect (doesn't prove it).
2. Accuracy CI [0.80, 0.90]? → True accuracy plausibly anywhere in range; small differences inside it may be noise.

TL;DR

Set α, compute p; p < α → reject H_0. p is not P(H_0 true). Report effect sizes with CIs; beware multiple testing.

2.12 Information theory: entropy, cross-entropy and KL divergence

What it is

- **Entropy** $H(p) = -\Sigma\, p(x) \log_2 p(x)$: average **uncertainty** (surprise) in bits. Fair coin 1 bit; 90/10 coin 0.469; certain outcome 0.
- **Cross-entropy** $H(p, q) = -\Sigma\, p(x) \log q(x)$: average surprise when reality follows p but you predict q. Always ≥ H(p), equal only when q = p.
- **KL divergence** $KL(p \,\|\, q) = \Sigma\, p(x) \log (p(x)/q(x)) = H(p, q) - H(p)$: *extra* surprise from using q. ≥ 0, 0 only when q = p, **not symmetric**.

Why it matters

- **Classification loss.** With a **one-hot** true distribution (probability 1 on the right class), cross-entropy = −log q(correct class). Averaged, this is **log loss**, minimised by logistic regression and nearly every neural classifier, including language models predicting the next **token** (word or word-piece).
- **Decision trees** pick splits that most reduce entropy (**information gain**).

- **KL** appears in variational autoencoders (networks that compress data and generate samples), **knowledge distillation** (a small model copying a big model's probabilities), and **RLHF** (RL from human feedback), where it keeps a fine-tuned language model close to its original.

Worked numbers

True p = [0.7, 0.2, 0.1], model q = [0.5, 0.3, 0.2] (base 2):

- $H(p) = 0.360 + 0.464 + 0.332 =$ **1.157 bits**
- $H(p, q) = -(0.7 \log_2 0.5 + 0.2 \log_2 0.3 + 0.1 \log_2 0.2) = 0.700 + 0.347 + 0.232 =$ **1.280 bits**
- $KL(p \| q) =$ **0.123 bits**

Log loss, labels [1, 0, 1, 1], predicted P(class 1) = [0.9, 0.2, 0.6, 0.99] (natural log): −¼ [ln 0.9 + ln 0.8 + ln 0.6 + ln 0.99] = **0.2123**.

```
manual = -np.mean(y_true * np.log(p_pred) + (1 - y_true) * np.log(1 - p_pred))
log_loss(y_true, p_pred)   # scikit-learn gives the same number
```

Output of `code/information_theory.py`:

```
entropy of fair coin            = 1.000 bits
entropy of biased coin 0.9/0.1  = 0.469 bits
entropy of certain              = 0.000 bits
entropy of fair 8-sided die     = 3.000 bits

H(p) = 1.157, H(p, q) = 1.280, KL(p||q) = 0.123, H(p) + KL = 1.280
KL(q||p) = 0.133  (KL is not symmetric)

log loss by hand = 0.2123 | sklearn = 0.2123
  true class predicted with p=0.99: loss = 0.010
  true class predicted with p=0.6 : loss = 0.511
  true class predicted with p=0.1 : loss = 2.303
  true class predicted with p=0.01: loss = 4.605
```

Log loss punishes **confident mistakes**: giving the true class 1% costs 460× more than giving it 99%.

Common mistakes

- Mixing log bases: $\log_2$ (bits) vs natural log (nats, used by ML libraries); only the scale changes.
- Treating KL as a distance: it is asymmetric and violates the triangle inequality (going A→C directly is never longer than via B).
- Predicting exact 0 or 1: log(0) = −∞, so libraries clip probabilities (e.g. to 1e-15 … 1 − 1e-15).

Self-check

1. Loss when the true class gets 0.5? → $-\ln 0.5 \approx 0.693$.

TL;DR

Entropy = uncertainty; cross-entropy = surprise predicting with q; KL = the gap. Classifiers minimise cross-entropy (log loss), which punishes confident errors hardest.

Chapter TL;DR

Area	Key idea	Where it shows up
Vectors	Data point = vector; dot product = multiply and sum	Linear models, embeddings, similarity search
Matrices	Dataset = matrix; predictions = Xw + b	Every model; neural-net layers; GPUs
Transpose / inverse / det	Swap axes; undo a matrix; det = 0 → no inverse	Normal equation, gradients, covariance
Eigen / SVD	Main directions and their strengths	PCA, compression, recommenders
Derivatives / gradients	Slope of the loss per parameter	How every model learns
Chain rule	Multiply slopes through nested functions	Backpropagation
Gradient descent	$w \leftarrow w - \eta \nabla L$; tune η, scale features	Training almost everything
Distributions	Bernoulli, binomial, normal; mean, variance	Labels, noise, A/B tests
Bayes	Posterior $\propto$ likelihood × prior; mind the base rate	Naive Bayes, rare-event precision
Statistics	Median vs mean, standardisation, CLT, $\sigma/\sqrt{n}$	Preprocessing, trusting results
Correlation ≠ causation	Confounders; experiments for causes	Interpreting models
Tests and CIs	p-value, α, CI, bootstrap; significance ≠ importance	Model comparison, A/B tests
Information theory	Entropy, cross-entropy, KL	Log loss, decision trees, LLM training